

第三、四章重写稿

本稿依据仓库中的整篇论文版本、单章版本、0511 版本、thesis-v2 版本以及用户提供的初稿 DOCX 重新整理。旧稿中第三章曾把任务域、上下文、工具、安全控制分别拆成多个二级节，优点是覆盖完整，问题是设计论述被切得较碎，容易与第四章实现内容重叠。本稿将第三章收束为场景、总体架构、核心组件、角色协作四个设计层次；第四章则集中说明 Crater 平台中的前后端集成、智能体服务、工具体系和评估模块实现。

写作来源对照

来源位置	原有写法	本稿取舍
docs/zh-CN/thesis-面向智算平台的运维智能体设计与实现-v2.md	第三章从运维对象、任务域、Mops 总体设计、任务分流、上下文、工具、安全控制展开；第四章按 Crater 基础能力、前端、Go 后端、Python Agent、工具、质量评估展开	保留任务对象、工具、安全和评估内容，但压缩三级目录。第三章改为设计逻辑，第四章承接实现细节
docs/zh-CN/thesis-面向智算平台的运维智能体设计与实现-draft-v3.md	结构与 v2 相近，文字更接近完整论文，但仍存在第三章和第四章边界交叉	吸收其任务分类、Mops 设计目标、工具数量和 Crater 说明，删去设计章中的具体接口和单智能体实现描述
docs/thesis-v2/chapter3-design.md 与 chapter4-implementation.md	单章稿强调四层架构、MASState、工具声明与执行分离、安全审计、系统实现	保留 MASState、角色视图、确认恢复和工具解耦等核心设计，避免按实现模块重复拆分
crater-agent/docs0511/chapter3-design.md 与 chapter4-implementation.md	单章稿较完整，包含角色协作、上下文、工具、安全、ER 模型、SSE、评估与巡检	作为本稿主要参考之一，但将 ER 模型、SSE、执行器、质量评估等内容集中到第四章
docs/0511/03-system-design.md 与 04-implementation.md	旧版强调创新点、意图路由、异构 Agent、工具双路由和审批链	仅保留可落地的设计思想，不继续使用过强的创新点叙述和不稳定术语
用户提供的 DOCX 初稿	第三章为多智能体协作框架设计，第四章为系统实现，包含 Crater 基础能力、工具数量、确认恢复和评测支撑	作为章节顺序和论文语气参考。根据新的目录要求重排为 3.1 至 3.5、4.1 至 4.4

复制进正式论文时，可以从第 3 章标题开始使用。

3 面向智算平台运维的多智能体框架设计

本章面向智算平台的真实运维场景，设计一种多智能体运维框架 Mops。Mops 为 Multi-agent Operations for Intelligent Computing Platform 的缩写，目标是在保留平台既有管理能力的基础上，使自然语言交互、平台观

测数据、运维工具、安全确认和结果评估形成可控的闭环。与第四章不同，本章关注框架为何这样划分、各组件之间如何约束以及任务如何在多智能体之间流转，不涉及具体接口、代码结构和部署细节。

智算平台运维任务的特殊性在于，运维对象既包括云原生基础设施，也包括人工智能训练任务、镜像环境、队列调度、存储网络和多用户资源配额。一个看似简单的用户请求往往需要跨越多个数据源。例如，用户询问作业为何一直处于等待状态，系统不能只查看作业状态，还需要结合队列容量、用户配额、节点资源、调度事件和近期任务变化综合判断。如果请求进一步转化为停止作业、重提作业、释放节点或处理审计项，则系统还必须引入权限校验、人工确认和操作后验证。因此，本文将 Mops 设计为以多角色协作为核心、以平台工具为证据来源、以确认恢复机制为安全边界的运维智能体框架。

3.1 场景描述与问题分析

智算平台的运维对象可以从资源、作业、环境和治理四个层面理解。资源层包括 GPU、CPU、内存、节点、队列、存储卷和网络链路，是平台承载训练任务的基础。作业层包括批处理训练作业、交互式开发环境、分布式训练任务、镜像构建任务和用户提交的自定义任务，是用户最直接感知的平台对象。环境层包括基础镜像、用户镜像、CUDA 与框架版本、镜像仓库、存储挂载、服务暴露和运行时依赖。治理层则包括用户账户、资源配额、审批工单、审计项、异常作业治理和管理员操作记录。

这些对象之间不是孤立关系。作业状态取决于调度器、节点资源、镜像拉取、存储挂载和用户配额；用户体验取决于前端页面、作业生命周期、日志可读性和问题反馈路径；管理员治理则需要把节点状态、资源利用率、用户行为、风险操作和审计结果关联起来。传统运维系统通常把这些能力分散在日志平台、监控面板、Kubernetes 命令行、平台后台和人工经验中，用户或管理员需要自行切换工具并拼接证据。对于智算平台而言，这种方式会带来三个突出问题。

第一，任务语义与平台对象之间存在转换成本。用户通常以自然语言描述问题，例如显存不够、队列卡住、镜像找不到、训练没速度、能否帮我停掉这个任务。系统需要把这类表达映射为平台中的具体对象、工具和风险等级。若缺少这种语义转换，智能体容易停留在泛化建议，无法真正解决平台问题。

第二，证据来源分散，单一工具难以支撑可靠判断。作业失败可能来自用户代码、镜像依赖、资源不足、节点异常、网络抖动或存储故障。仅凭日志片段或监控曲线很容易误判。可靠的运维回答必须先收集证据，再给出结论，并说明结论与证据之间的关系。Mops 因此不能只做聊天问答，而要能够组织多轮观察和复核。

第三，运维操作具有真实副作用。停止作业、重提作业、删除 Pod、节点隔离、队列治理和审计处理都会改变平台状态，且可能影响其他用户或集群稳定性。智能体在这类场景中不能直接执行模型生成的操作，而应将写操作纳入权限控制、人工确认、审计记录和操作后验证流程。

据此，本文将智算平台运维任务划分为六类。第一类是平台使用引导，回答用户关于作业提交、镜像选择、日志查看和交互式环境使用的问题。第二类是状态查询，围绕作业、镜像、队列、配额、节点和存储提供事实性信息。第三类是故障诊断，针对作业失败、等待、性能异常、分布式训练异常和镜像构建失败进行证据收集和根因分析。第四类是资源治理，面向管理员发现空跑作业、异常占用、节点异常、队列拥塞和资源浪费问题。第五类是受控执行，涉及停止、重提、删除、扩缩容、节点隔离和审计处理等高风险操作。第六类是任务型流程，包括审批评估、巡检报告和面向固定业务流程的专门处理。

上述任务可以形式化表示为

$$T = \langle U, O, C, G, R, H \rangle$$

其中，\$U\$ 表示用户与角色身份，\$O\$ 表示目标对象集合，\$C\$ 表示页面、会话和平台状态上下文，\$G\$ 表示用户意图或任务目标，\$R\$ 表示风险等级，\$H\$ 表示历史交互和已执行动作。该表示强调，运维任务不是单轮文本输入，而是带有身份、对象、状态和风险约束的过程。Mops 的设计目标也由此确定为五点：能够识别

多类型运维任务，能够使用真实平台工具收集证据，能够在不同智算平台之间迁移工具适配，能够对写操作和管理员操作建立可信边界，能够记录过程并支撑后续评估。

图 3.1 给出了智算平台运维任务对象与问题之间的关系。该图用于说明 Mops 面向的是平台运行过程中的综合运维任务，而不是单一故障问答。

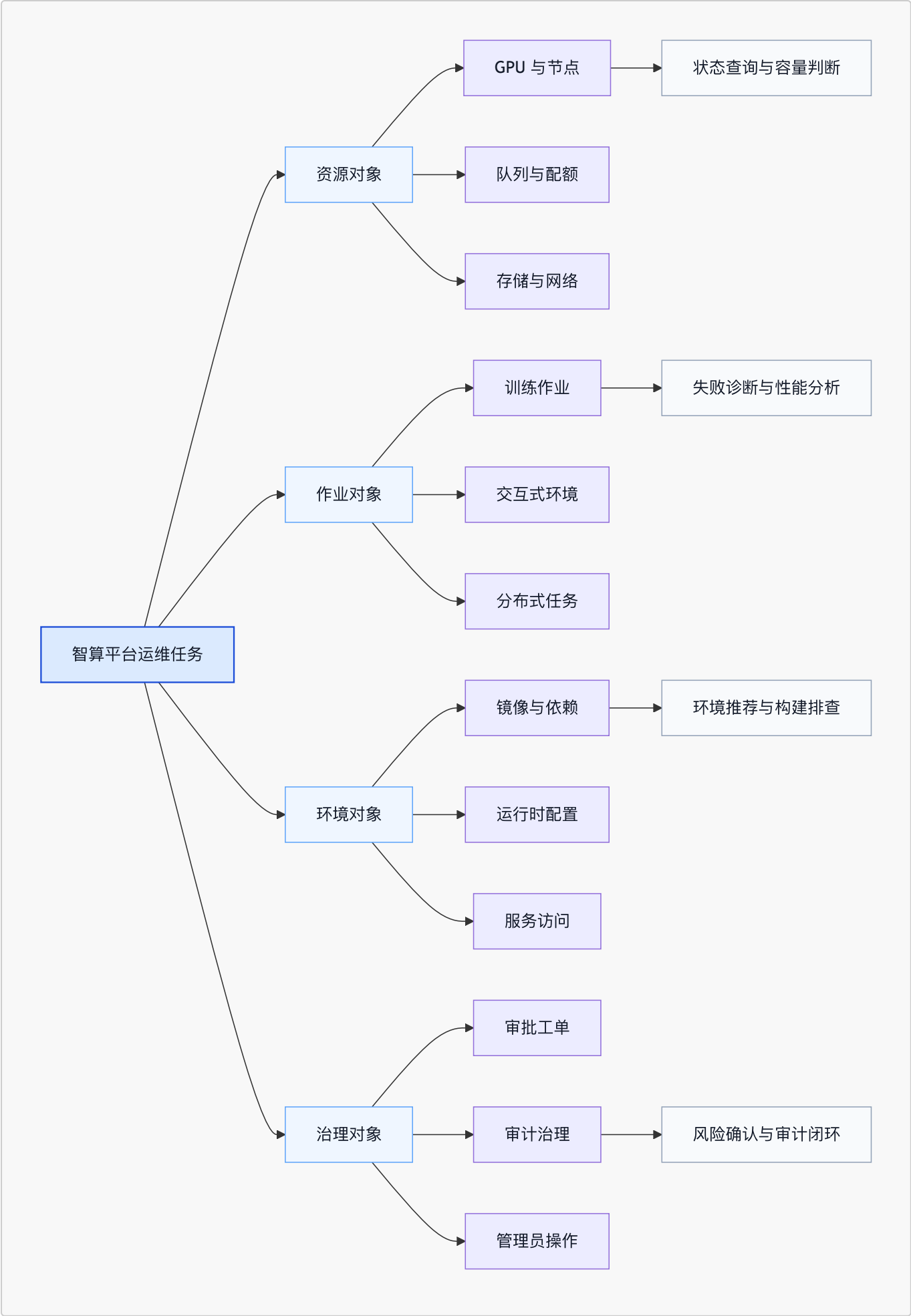


图 3.1 智算平台运维对象与任务关系

3.2 总体架构

Mops 的总体架构围绕用户交互、任务编排、智能体协作、工具适配与平台基础设施展开。为避免设计章节与实现章节混杂，本节只描述框架的逻辑层次和信息流。具体到 Crater 平台的前端组件、后端接口、数据表和服务部署，将在第四章展开。

Mops 的输入来自两类入口。一类是用户或管理员在平台页面中的自然语言请求，另一类是平台定时任务或业务流程触发的运维请求。前者强调多轮对话和页面上下文，后者强调固定任务结构和可审计结果。框架接收请求后，首先构造任务上下文，包括用户身份、账户范围、页面对象、会话历史、可用工具集合和是否存在待确认操作。随后由协调角色完成意图识别和任务分流，判断任务属于帮助、查询、诊断、治理、审批、巡检还是受控执行，并决定是否进入多智能体协作链。

在核心协作链中，协调器负责整体调度，规划器负责将任务拆分为可执行步骤，探索器负责调用只读工具收集证据，执行器负责处理受控动作，验证器负责检查证据、结论和操作结果是否一致。任务型智能体不取代这五个协作角色，而是作为特定业务流程的入口或专家能力存在。例如，帮助型智能体适合页面引导，审批智能体适合工单裁决，巡检流水线适合周期性健康报告。这种设计使 Mops 同时具备通用协作能力和面向业务流程的专门处理能力。

工具层是 Mops 连接平台事实的边界。工具声明描述工具名称、用途、参数、返回语义、风险等级和允许角色；执行适配负责把工具调用映射到不同平台的接口、SDK、Kubernetes API、Prometheus、存储系统或离线评测快照。智能体不直接依赖某个平台内部实现，而只依赖统一的工具语义。这样既可以在 Crater 中调用现有功能，也为迁移到其他智算平台保留空间。

安全与评估机制贯穿整个框架。对于只读工具，系统强调权限过滤、结果压缩和证据引用；对于写操作，系统强调最小权限、人工确认、暂停等待、确认后恢复和审计记录；对于最终回答，系统强调事实支撑、风险提示和质量评估。Mops 的总体执行原则可以概括为三条：先收集证据，再形成判断；先确认风险，再执行操作；先复核结果，再给出结论。

图 3.2 展示了 Mops 的总体架构。图中从上到下依次体现交互入口、任务编排、角色协作、工具适配和平台能力，安全控制与质量评估作为横向机制覆盖全流程。

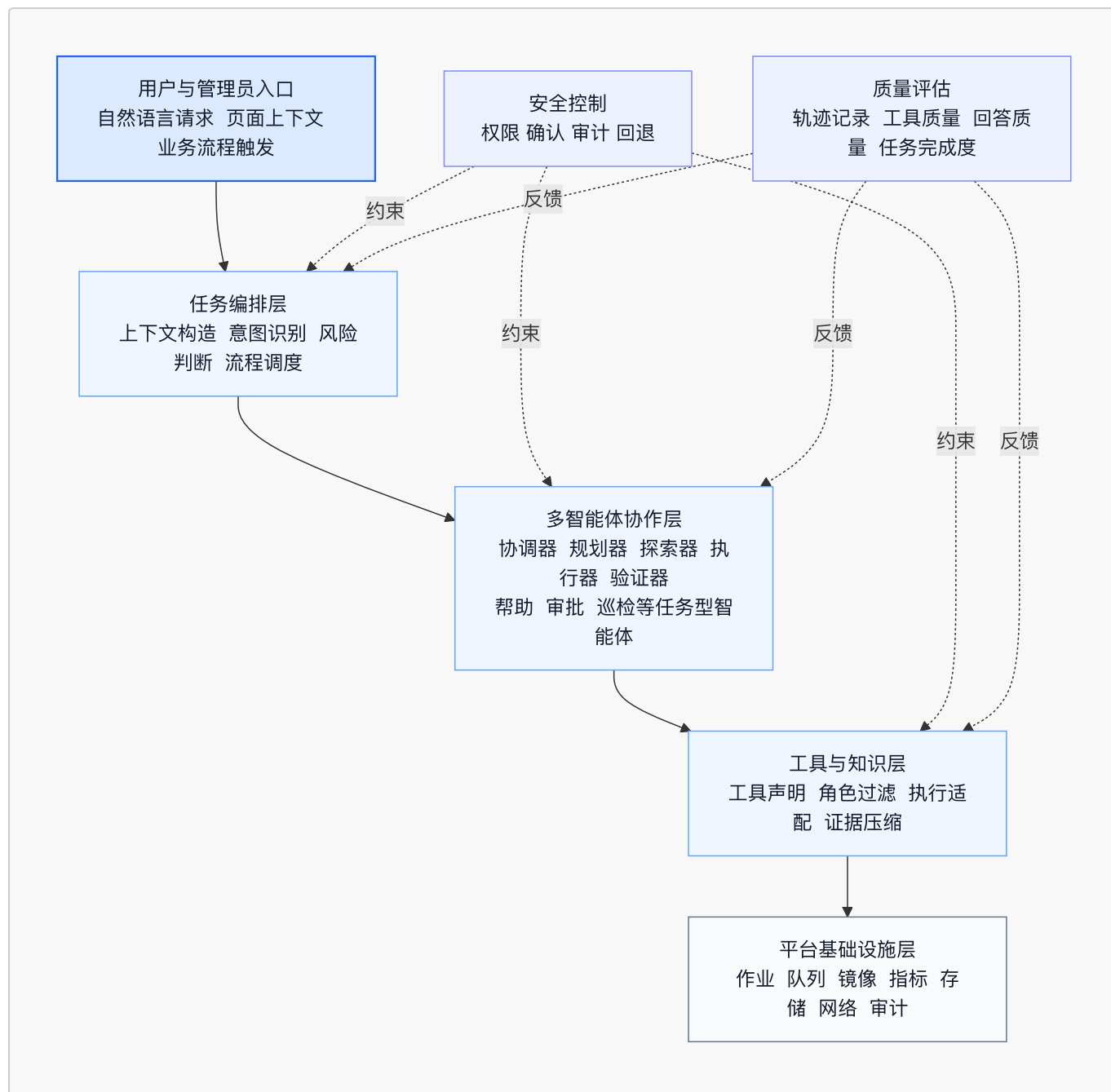


图 3.2 Mops 总体架构

从框架边界看，Mops 不试图替代智算平台的调度器、存储系统、监控系统或权限系统，而是在这些既有系统之上构建一层面向任务的智能运维编排。平台继续负责真实数据、真实权限和真实操作，Mops 负责把用户意图转化为有证据、有约束、可审计的运维过程。这样的边界划分有利于工程落地，也避免智能体绕过平台原有治理机制。

3.3 智能体核心组件设计

智能体核心组件用于支撑多角色协作过程中的上下文保存、工具调用、安全控制、运行约束和质量评估。本节不再继续拆分三级标题，而是按照组件职责进行论述，以突出各组件之间的关系。

记忆机制用于保持任务过程的连续性。 智算平台运维请求通常不是孤立问答，而是与用户身份、页面对象、历史对话和前序操作有关。Mops 将记忆划分为四类。第一类是身份记忆，保存用户、账户和角色范围，用于决定数据可见性和工具权限。第二类是页面记忆，保存当前页面、作业名、节点名、镜像名或审计项等对象，使用户可以使用这个任务、刚才那个节点等上下文表达。第三类是会话记忆，保存多轮对话中已经确认的事实、

用户补充信息和系统给出的建议。第四类是任务记忆，保存当前目标、计划、证据、动作、待确认项和恢复状态。

记忆机制的关键不在于保留尽可能多的原始文本，而在于把可用信息组织为不同角色能够消费的状态。规划器需要知道目标和已有证据，探索器需要知道缺口和候选工具，执行器需要知道待执行动作及其风险，验证器需要知道证据是否足以支撑结论。若所有角色都读取完整上下文，不仅会增加上下文负担，还可能导致不同角色重复调用工具或忽略安全边界。因此，Mops 采用共享状态与角色视图相结合的方式：共享状态保存完整任务过程，角色视图只向某个角色暴露完成其职责所需的信息。

工具机制用于把智能体推理限定在平台事实之内。智能体若只依赖模型内部知识，难以判断真实作业状态、队列容量或节点异常。因此，Mops 将工具作为连接平台事实的唯一执行通道。工具设计采用声明与执行分离的方式。声明层描述工具语义，包括工具能解决什么问题、需要哪些参数、返回什么类型的证据、属于只读还是写操作、哪些角色可以使用。执行层负责适配不同智算平台，例如在 Crater 中调用后端接口，在集群环境中查询 Kubernetes API 和 Prometheus，在存储诊断中读取 Ceph 或 PVC 信息，在离线评测中读取预先记录的工具体快照。

这种分离有两个作用。一方面，智能体只面向统一工具语义，不需要了解平台内部接口差异，从而减少迁移成本。另一方面，工具执行可以独立实施权限校验、日志记录、超时控制和结果格式化，避免模型直接接触不可控的底层操作。对于只读工具，系统可以自动执行并返回证据；对于写操作，系统必须进入暂停、等待确认和恢复流程。也就是说，当执行器提出一个有副作用的动作时，框架并不立即完成操作，而是生成确认请求并中断当前任务。用户确认后，平台执行真实操作，Mops 再根据确认结果恢复原任务状态并继续验证。这一机制保证了写操作不会脱离用户授权和审计链路。

安全机制用于限制智能体可见范围和可执行范围。智算平台具有多用户和多角色特征，普通用户、课题组管理员和平台管理员能够看到的数据与能够执行的操作不同。Mops 将安全边界设计为多层控制。第一层是用户身份与账户范围控制，确保工具返回的数据不越权。第二层是角色级工具控制，规划器、探索器和验证器主要使用只读工具，执行器才允许发起确认型写操作。第三层是工具风险分级，节点隔离、批量停止、删除资源、执行管理员命令等操作属于高风险动作，需要更严格的确认和审计。第四层是人机协同审批，Human in the Loop 在本文中简称 HITL，用于把关键决策权保留给用户或管理员。

安全机制不是简单地在最终执行前弹出确认框，而是贯穿任务识别、计划生成、工具过滤、动作生成和结果复核全过程。协调器在任务入口判断风险；规划器在拆解步骤时避免把未确认的写操作当作事实；执行器必须以明确目标和最小必要参数发起操作；验证器在操作后检查执行结果是否与用户意图一致。若用户拒绝确认，任务应转为解释拒绝结果和给出替代建议，而不是反复请求执行同一动作。

运行时约束用于防止无边界推理和无效循环。多智能体系统虽然提高了任务分解和复核能力，但也带来循环调用、重复取证、回答延迟和上下文膨胀等问题。Mops 将运行时约束归入 runtime harness，即任务执行期间的保护边界。该边界包括协调轮数限制、子角色调用限制、工具超时、重复工具检测、无进展收敛和异常回退。若系统在连续步骤中没有获得新证据，协调器应收束为已有证据下的保守结论；若某个工具失败，系统应判断是否可换用其他证据来源；若任务目标不明确，系统应要求用户补充关键信息，而不是盲目执行高风险动作。

运行时约束还承担故障降级职责。对于只读诊断任务，若部分数据源不可用，系统可以明确说明证据缺口，并基于已有数据给出有限结论。对于写操作任务，若确认状态丢失或执行结果不完整，系统应停止继续操作，提示用户查看平台状态或由管理员复核。对于复杂多轮任务，若达到轮次上限，系统应输出当前证据、未完成事项和建议下一步，而不是生成确定性过强的结论。

评估机制用于度量任务质量并反向改进框架。运维智能体的质量不能只看回答是否流畅，更要看工具选择是否正确、证据是否充分、根因判断是否准确、风险控制是否合规、操作结果是否可追踪。Mops 因此在设计上保

留完整轨迹，包括用户请求、路由判断、角色动作、工具调用、确认记录、最终回答和用户反馈。基于这些轨迹，可以从任务完成度、工具调用质量、事实一致性、安全合规性和用户可用性等维度评估系统表现。

评估机制与前述组件存在双向关系。记忆机制提供可复现的上下文，工具机制提供可核验的证据，安全机制提供确认和审计记录，运行时约束提供终止原因和失败类型。评估结果则可以反过来发现工具描述不清、角色边界不稳、确认流程过长或回答证据不足等问题。由此，Mops 不是一次性生成回答的系统，而是能够通过过程记录和质量评估持续改进的运维框架。

图 3.3 展示了核心组件之间的关系。记忆、工具、安全、运行时约束和评估不是并列孤岛，而是共同约束一次任务从输入到输出的全过程。

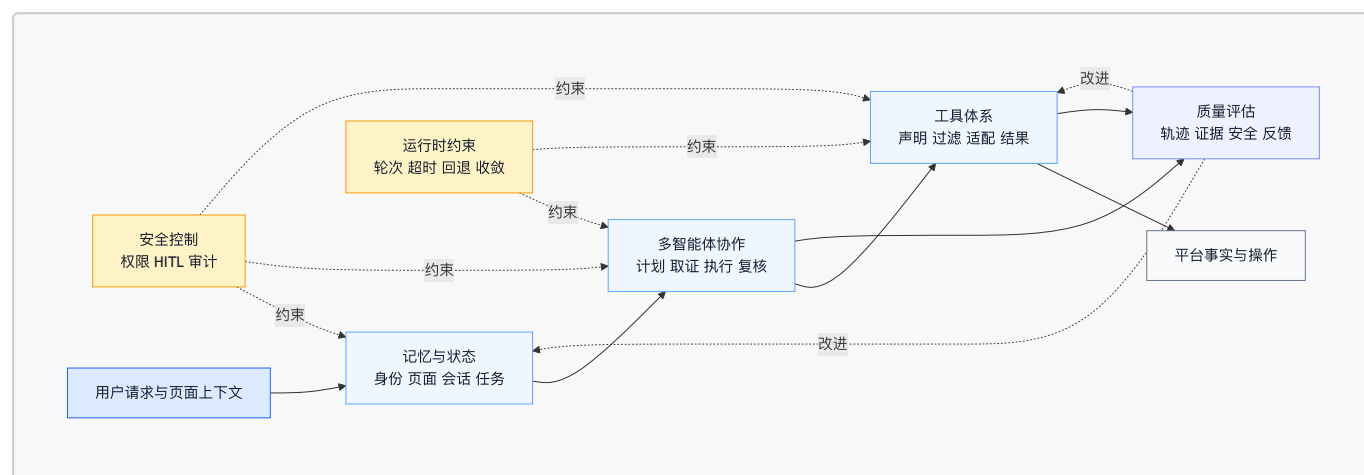


图 3.3 智能体核心组件关系

3.4 多智能体角色与协作机制设计

Mops 的多智能体设计并不是简单增加多个模型调用，而是将运维任务中的关键职责分离，使每个角色围绕较明确的判断边界工作。智算平台运维同时需要理解用户意图、拆解调查步骤、调用平台工具、处理高风险动作和复核最终结论。若由单一角色完成全部工作，容易出现计划与执行混杂、证据不足就下结论、重复调用工具或绕过确认流程等问题。多角色协作的价值在于把任务中的判断点显式化，使系统能够在每个阶段留下可追踪记录。

3.4.1 智能体角色定义

Mops 的通用协作链包含五个角色。协调器负责入口判断和流程控制，是多智能体状态流转的主要控制者。协调器读取用户请求、页面上下文、历史状态和风险信息，判断当前应进入规划、探索、执行、验证、澄清还是总结。它不负责替代其他角色完成所有任务，而是控制任务的下一步方向。

规划器负责把用户目标拆分为可执行的调查或处理步骤。对于诊断任务，规划器需要确定应先查看作业详情、日志、事件、指标还是队列状态；对于治理任务，规划器需要明确筛查对象、判断条件和风险边界；对于受控执行任务，规划器需要把执行前检查、确认动作和执行后验证分开。规划器的输出应是任务步骤和候选工具，而不是最终结论。

探索器负责收集证据。它根据规划器给出的方向调用只读工具，整理事实、证据和仍然缺失的信息。探索器的核心要求是基于平台真实数据回答问题，避免用常识替代观测结果。对于作业诊断，探索器应关联日志、事件、退出码、指标和历史相似失败；对于集群治理，探索器应关关节点状态、资源利用率、任务分布和审计数据。

执行器负责处理需要改变平台状态的动作。执行器只在目标明确、权限允许、风险已识别且必要证据充分时发起写操作。执行器输出的不是直接完成操作的承诺，而是带有对象、参数、风险摘要和确认要求的动作请求。用户确认后，执行器再接收平台执行结果，并将结果写入共享状态。这样可以避免模型把建议性文本误当作真实操作。

验证器负责复核证据、动作和最终回答。它检查诊断结论是否被工具结果支撑，检查写操作是否存在真实确认和执行结果，检查回答是否夸大确定性或遗漏风险。验证器不以重新完成任务为目标，而是作为质量门控角色，对不充分的结论提出补证、澄清或降级建议。

除五个通用角色外，Mops 还设置任务型智能体。帮助型智能体面向平台使用引导，适合处理页面说明、作业提交步骤和镜像选择等低风险问题。巡检型智能体面向周期性或触发式健康检查，通常以结构化平台数据为输入，生成集群状态、风险项和治理建议。工单与审批型智能体面向资源延期、作业锁定、审计处理等制度化流程，重点在于根据平台记录和规则给出可审计建议。它们属于 Mops 的专家任务框架，用于处理具有固定输入输出形态的任务，而不是替代通用五角色协作链。

表 3.1 总结了各角色的职责、输入和输出。角色划分的原则是减少职责重叠，使每个阶段都能回答一个清晰问题：是否进入任务、如何拆解任务、需要哪些证据、能否执行动作、结论是否可靠。

角色	主要职责	主要输入	主要输出
协调器	识别意图、判断风险、控制状态流转	用户请求、上下文、共享状态	下一阶段决策、澄清请求、总结指令
规划器	拆解任务步骤、选择候选工具	任务目标、已有证据、风险等级	调查计划、候选工具、风险提示
探索器	调用只读工具收集证据	计划步骤、工具集合、角色视图	事实摘要、证据列表、待解决问题
执行器	发起受控动作并处理确认结果	目标动作、权限信息、证据状态	确认请求、执行结果、动作状态
验证器	复核证据、结论和操作结果	证据、计划、执行结果、最终草稿	通过、补证、降级或风险提示
帮助型智能体	平台使用引导和低风险问答	页面信息、平台知识	操作说明和使用建议
巡检型智能体	健康检查与治理报告	节点、作业、资源和审计数据	巡检报告和风险项
工单与审批型智能体	审批评估和制度化流程处理	工单信息、历史记录、平台状态	通过、驳回或人工升级建议

3.4.2 多智能体协作机制设计

多智能体协作的核心是共享状态 MASState。MASState 保存一次任务从入口到结束的结构化过程，包括目标、路由判断、计划、观测、证据、动作、待确认项、工具记录、验证结果和终止原因。它不是简单的聊天记录，而是任务执行的状态容器。每个角色只读取与自身职责相关的状态视图，并把结果写回共享状态。这样既可以让角色之间共享事实，又可以防止角色越权读取或修改不必要的信息。

可将一次任务的状态流转表示为

$$S_{t+1} = \Delta(S_t, r_i, a_i)$$

其中，\$S_t\$ 表示当前 MASState，\$r_i\$ 表示参与当前阶段的角色，\$a_i\$ 表示该角色产生的计划、观察、动作、验证意见或总结。状态转移函数 δ 的含义是：角色不能直接改变真实平台状态，只能通过框架允许的动作改变任务状态；真实平台状态的改变必须经过工具执行和确认机制。通过这一约束，Mops 将模型输出和平台副作用分离开来。

典型协作流程如下。第一步，系统构造初始状态，将用户请求、身份、页面对象和历史上下文写入 MASState。第二步，协调器进行入口判断。如果请求属于帮助或固定审批流程，则转入相应任务型智能体；如果请求属于复杂查询、诊断、治理或受控执行，则进入通用五角色协作链。第三步，规划器生成调查计划，明确所需证据和候选工具。第四步，探索器调用只读工具，收集事实并更新观测结果。第五步，协调器根据观测结果判断是否需要继续补证、发起执行、请求用户澄清或进入总结。第六步，若涉及写操作，执行器生成确认请求，任务暂停并等待用户确认。第七步，用户确认或拒绝后，系统恢复原 MASState，将确认结果与待执行动作匹配，再由协调器决定是否进行执行后验证。第八步，验证器复核证据和结论，最后由协调器组织最终回答。

在这个过程中，协调器控制状态机的主要流转，验证器提供质量门控反馈，执行器控制待确认动作的产生，用户确认控制写操作能否继续。规划器和探索器不应直接触发写操作，验证器也不应替代执行器执行动作。这种边界可以减少角色之间的职责重叠，使每次任务流转都能在状态中解释。

图 3.4 展示了多智能体协作与状态流转关系。图中虚线表示安全和验证反馈，实线表示主要任务流。

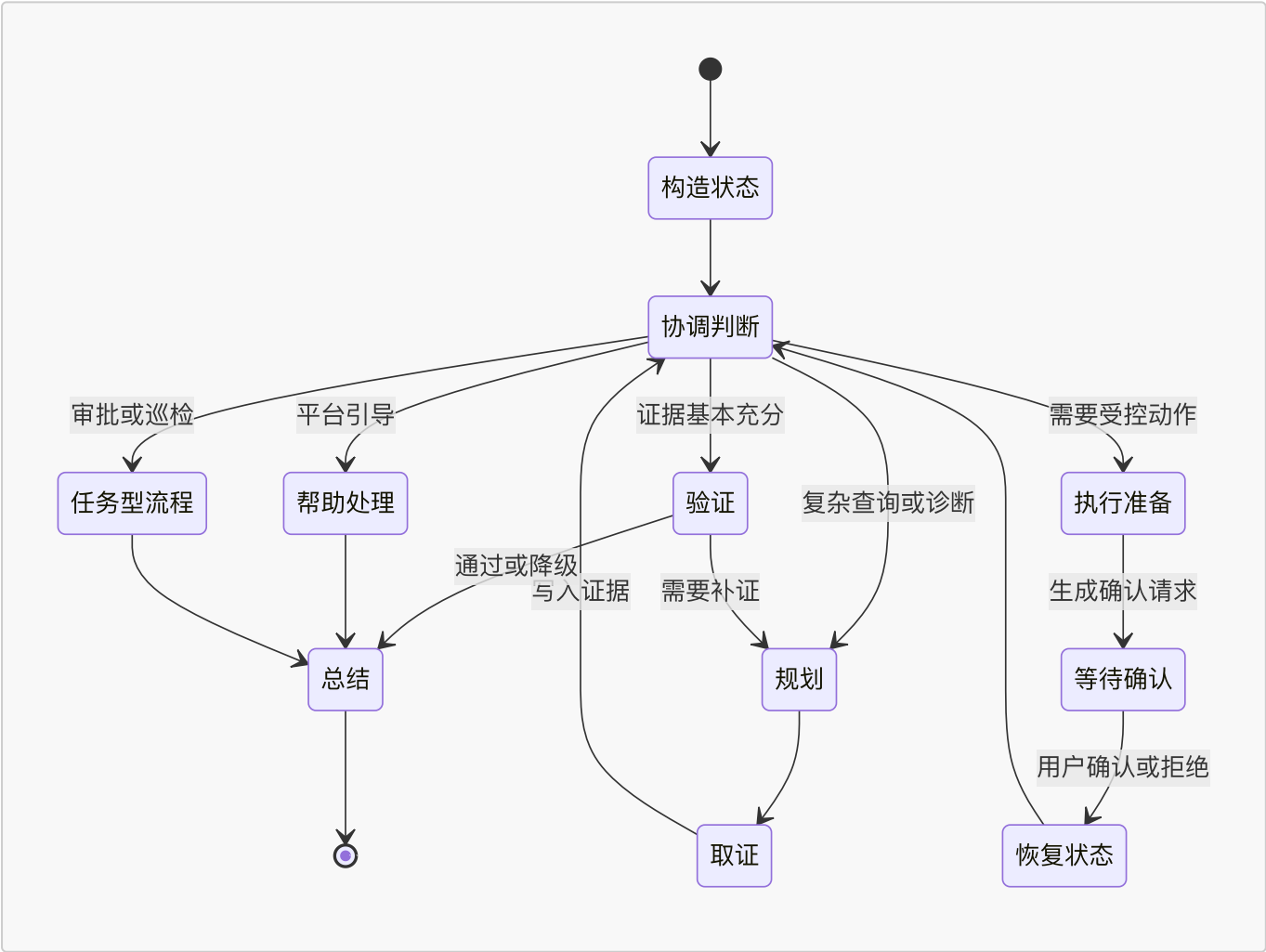


图 3.4 多智能体状态流转

为了避免协作链过长，Mops 在状态机中设置收敛条件。若任务已经获得足够证据，协调器应进入验证和总结，而不是继续取证。若多轮取证没有新增事实，系统应报告证据不足并给出下一步建议。若用户请求本身缺

少目标对象或必要参数，系统应优先澄清。若写操作等待确认，任务应暂停而非继续推理。若确认结果已经返回，系统应恢复原任务并复核结果，而不是重新发起同一写操作。

这种协作机制使 Mops 可以在不同任务之间保持统一框架。对于普通查询，流程可能只经过协调、取证、验证和总结。对于故障诊断，流程会多次在规划、取证和协调之间流转。对于受控执行，流程会显式经过执行准备、等待确认和恢复状态。对于审批和巡检，则由任务型智能体完成专门处理，同时仍受权限、审计和评估机制约束。

3.5 本章小结

本章从智算平台运维对象和任务特征出发，提出了 Mops 多智能体运维框架。首先，本文分析了智算平台中资源、作业、环境和治理对象之间的耦合关系，指出智能运维任务需要同时处理语义理解、多源证据、平台工具和操作风险。其次，本文给出 Mops 的总体架构，将用户交互、任务编排、多智能体协作、工具适配、平台基础设施、安全控制和质量评估组织为统一框架。再次，本文围绕记忆、工具、安全、运行时约束和评估五类核心组件，说明系统如何保持任务连续性、限制工具边界、实施人机协同确认并记录可评估轨迹。最后，本文定义了协调器、规划器、探索器、执行器和验证器五个通用协作角色，以及帮助、巡检、工单与审批等任务型智能体，并基于 MASState 设计了多智能体状态流转机制。

第三章的重点是抽象设计。它回答的是 Mops 为什么需要多角色协作、为什么需要共享状态、为什么要把工具声明与执行适配分离，以及为什么写操作必须通过确认和恢复机制。下一章将转向工程实现，说明上述设计如何在 Crater 智算平台中落地为前端交互、Go 后端服务、Python 智能体服务、工具执行体系和质量评估模块。

4 智算平台运维智能体系统实现

本文以 Crater 智算平台作为 Mops 的原型实现环境。Crater 面向多用户人工智能研发场景，已经具备作业管理、交互式开发、镜像管理、队列调度、资源配额、监控审计和管理员治理等基础能力。Mops 的实现并不是重写这些平台能力，而是在 Crater 现有前端、后端和集群接口之上增加智能体服务，使自然语言请求能够调用已有平台能力，并在必要时进入确认、恢复和审计流程。

图 4.1 用黑白思维导图概括 Crater 的能力底座。该图可以放在第四章开头，用两三句话承接第三章抽象框架与后文系统实现。

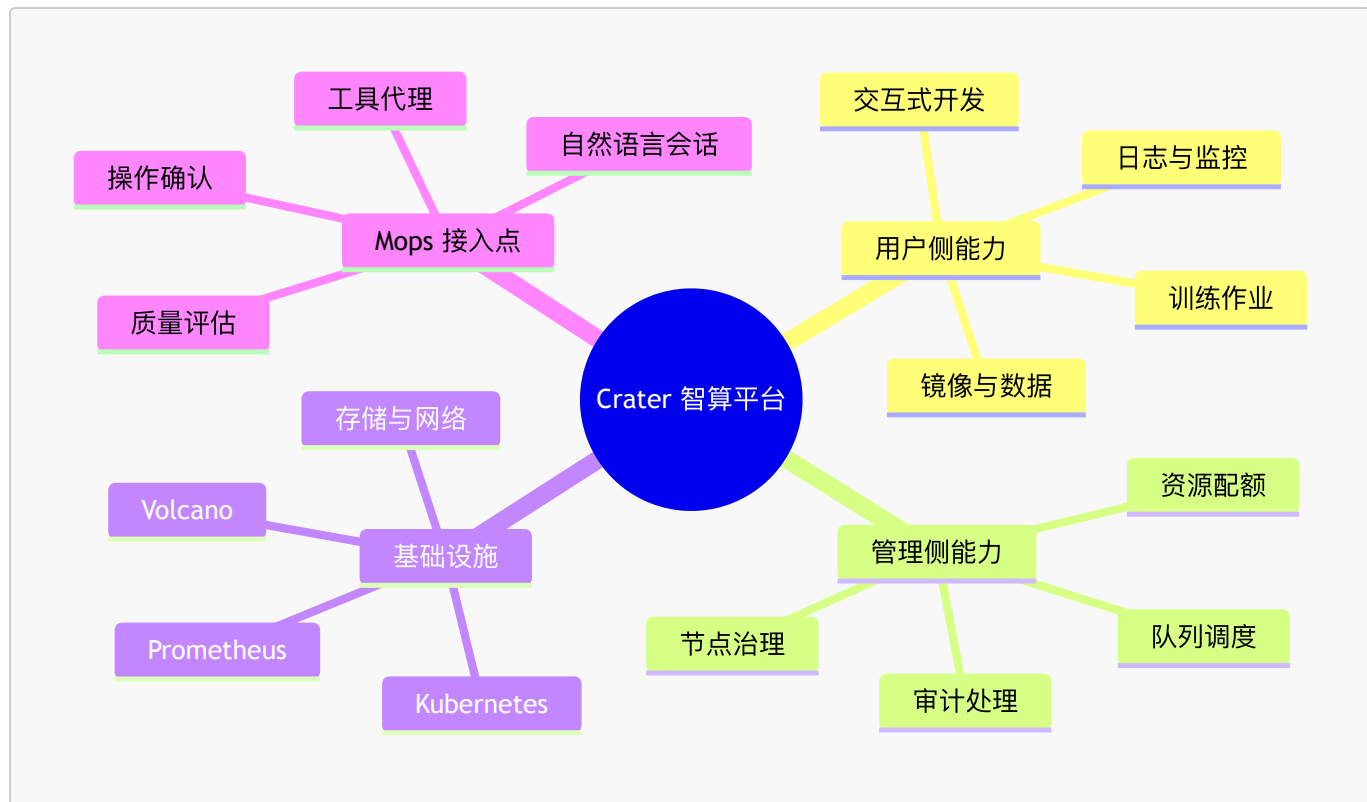


图 4.1 Crater 能力底座与 Mops 接入位置

4.1 系统整体架构

Mops 在 Crater 中采用前端、Go 后端和 Python 智能体服务分离的实现方式。前端负责会话入口、事件流展示、工具调用卡片、参数确认卡片和管理员审计页面。Go 后端负责用户鉴权、会话持久化、上下文构造、工具代理、权限校验、确认流处理和审计记录。Python 智能体服务负责多智能体编排、上下文组织、工具声明加载、工具调用决策、质量评估和巡检流水线。底层平台能力仍由 Crater 后端、Kubernetes、Prometheus、镜像仓库和存储系统提供。

系统请求从前端进入后，Go 后端首先创建或读取会话，写入用户消息，并结合登录用户、账户、页面对象和历史消息构造请求上下文。随后后端以流式方式调用 Python 智能体服务。Python 服务在多智能体编排过程中产生思考事件、角色事件、工具调用事件、确认请求和最终回答。Go 后端将这些事件转换为前端可消费的服务端事件流，同时把消息、工具调用、运行事件和最终回答写入数据库。若智能体调用只读工具，工具请求由 Python 服务发起并经由 Go 后端或本地适配器执行；若调用写工具，系统生成确认请求并暂停任务。用户在前端确认后，Go 后端执行真实操作，再通过恢复接口把结果交回 Python 服务继续原任务。

这种架构体现了第三章中的边界划分。前端不直接执行平台操作，Python 智能体服务不绕过 Crater 的权限系统，Go 后端不承担模型推理职责，底层集群组件不感知自然语言交互。各模块通过明确接口协作，使智能体能力可以作为平台插件接入，同时保留 Crater 原有的安全和审计机制。

图 4.2 展示了系统整体实现架构。图中 Mops 智能体服务位于前后端和平台基础设施之间，通过工具代理和本地适配器访问真实平台能力。

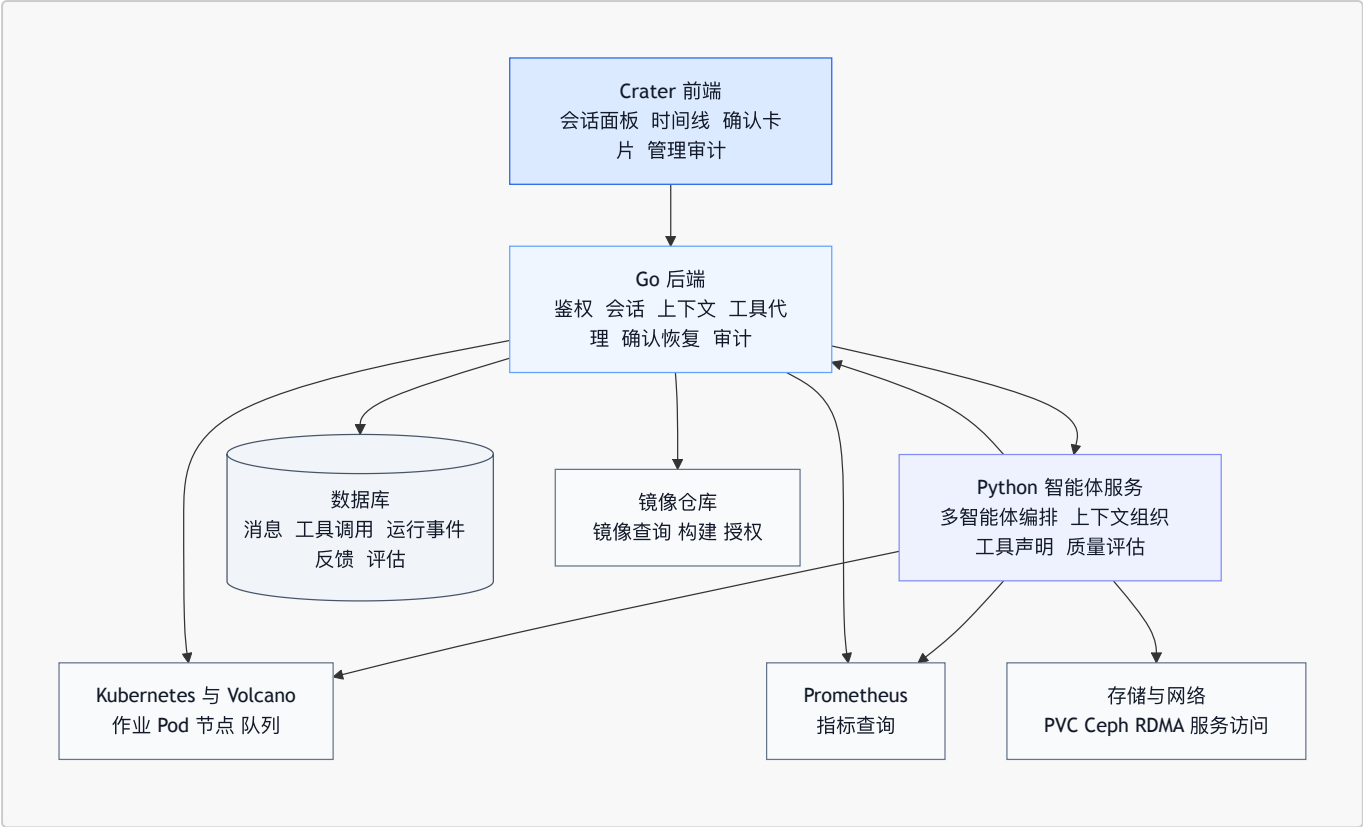


图 4.2 Mops 系统整体实现架构

4.2 前后端集成实现

前端集成主要围绕对话入口、过程可视化和确认交互展开。普通用户可以在作业、镜像、队列或帮助页面发起会话，页面会将当前对象信息随请求传给后端。例如在作业详情页中，前端可以附带作业名、作业类型、当前状态和页面路由；在管理员审计页中，前端可以附带审计项或集群对象信息。这样用户不必在每次提问中重复输入完整对象标识，系统也能把自然语言中的这个作业映射到明确平台对象。

对话过程通过事件流展示。前端不仅显示最终回答，还展示多智能体角色状态、工具调用结果和确认等待状态。对于只读工具，前端以工具卡片展示调用名称、参数摘要、执行状态和结果摘要，便于用户理解回答来源。对于写操作，前端展示确认卡片，列出操作对象、风险说明、参数表单和确认按钮。用户可以批准、拒绝或修改部分参数，后端据此执行真实操作或终止该动作。管理员页面进一步提供会话检索、工具调用轨迹、运行事件和质量评估结果，用于后续审计和问题排查。

Go 后端是前端与智能体服务之间的可信中介。它在会话开始时负责鉴权，确认用户身份和账户范围；在请求转发时负责构造上下文，包括用户信息、页面信息、历史消息、历史工具调用和可续接状态；在工具调用时负责权限校验和平台数据访问；在确认流中负责记录确认状态、执行真实操作、写入操作日志并触发恢复。后端提供的核心接口可以概括为四类：会话接口、工具执行接口、确认恢复接口和管理审计接口。

表 4.1 概括了 Crater 原有能力与 Mops 新增集成点之间的关系。可以看出，Mops 主要复用平台已有对象和接口，在其上增加自然语言入口、上下文注入、工具化封装和审计可视化。

Crater 已有能力	Mops 新增集成点	实现作用
作业管理与日志查看	作业详情、事件、日志和诊断工具	支撑作业查询和失败诊断
队列调度与资源配额	队列状态、容量、配额检查工具	支撑排队原因分析和资源建议
镜像管理与构建	镜像查询、构建详情、授权工具	支撑镜像推荐和构建排查

Crater 已有能力	Mops 新增集成点	实现作用
管理员治理	节点、审计、异常作业和批量治理工具	支撑巡检、审计处理和高风险操作
用户与账户权限	工具权限过滤和确认流	防止越权查询和越权执行
平台审计记录	会话、工具调用、运行事件和质量评估	支撑可追溯分析和离线评测

在数据持久化方面，后端保存智能体会话、消息、工具调用、运行事件、用户反馈和质量评估记录。会话表记录用户、账户、来源、页面上下文和最近编排模式；消息表记录用户与助手的对话内容；工具调用表记录工具名称、参数、结果、角色、执行来源、耗时和确认结果；运行事件表记录多智能体协作过程中的语义事件；反馈和质量评估表用于后续人工反馈与模型评估。这些数据共同构成 Mops 的审计链，也为第五章实验与评估提供样本基础。

前后端确认恢复流程如图 4.3 所示。该流程对应第三章提出的暂停、等待确认和恢复机制。

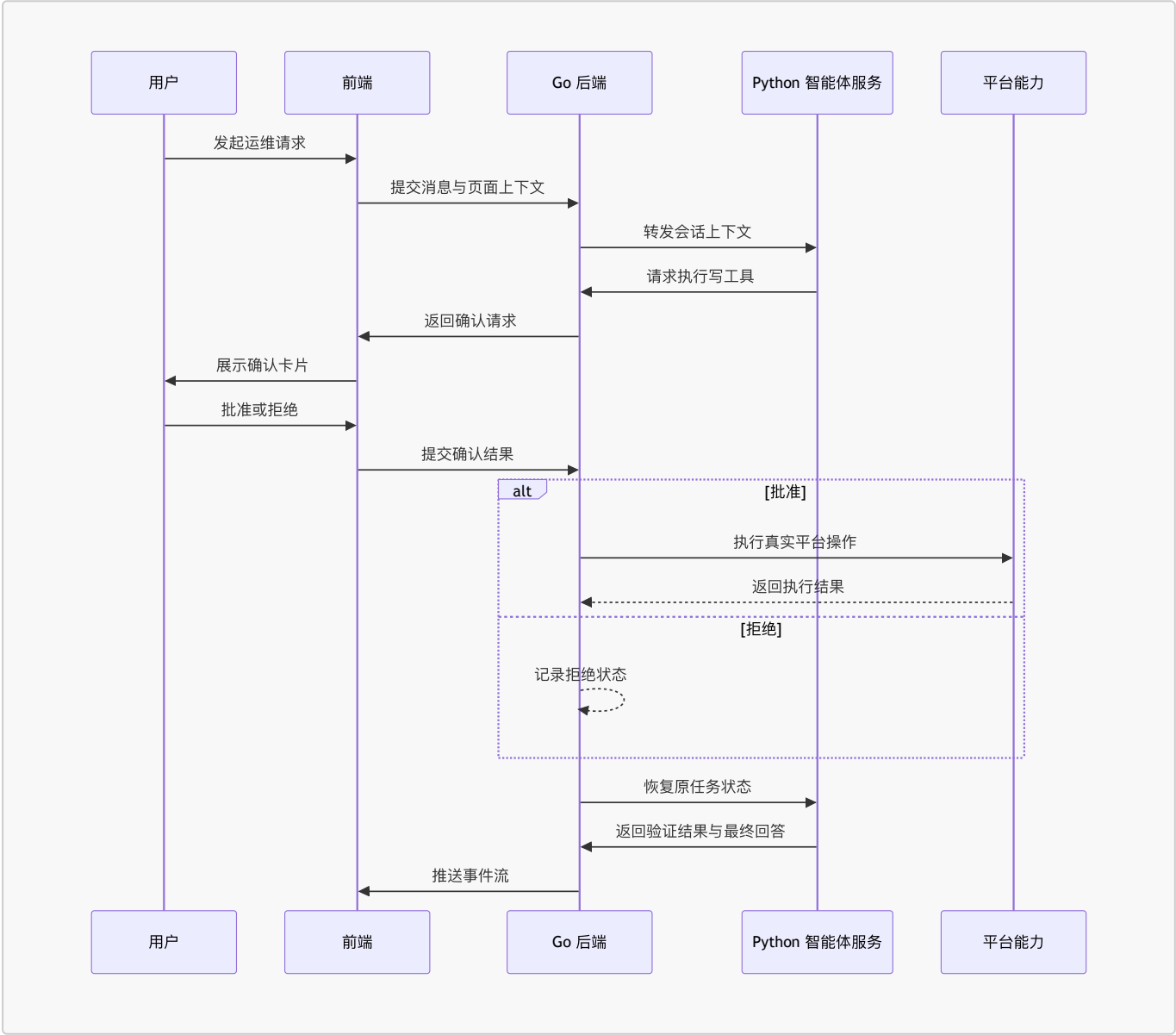


图 4.3 工具确认与恢复流程

4.3 智能体服务实现

Python 智能体服务是 Mops 的主要编排模块。服务对外提供流式会话接口、审批准评估接口、巡检流水线接口、质量评估接口和内部工具接口。其内部由上下文组织、多智能体编排、工具执行、运行时约束、任务型智能体和评估模块组成。

上下文组织实现。 智能体服务接收后端传入的会话请求后，首先构造统一上下文。上下文包括用户身份、账户范围、页面信息、历史消息、历史工具调用、待确认项、恢复结果和可用工具摘要。随后服务把这些信息写入 MASState，并为不同角色生成状态视图。协调器视图包含用户目标、路由判断、风险等级和当前进展；规划器视图包含目标、已有观测和候选工具；探索器视图包含计划步骤、开放问题和只读工具；执行器视图包含待执行动作、权限信息和确认状态；验证器视图包含证据、执行结果和最终回答草稿。

上下文组织还包含压缩和过滤。工具返回结果可能包含大量日志、指标序列或 Kubernetes 对象字段，不能全部进入模型上下文。系统会保留与任务相关的摘要、关键字段、异常片段和可引用证据，并过滤与当前角色无关或超出权限范围的信息。这样既降低上下文负担，也减少无关信息干扰判断。

运行时约束实现。 智能体服务在编排过程中通过配置化参数控制任务边界，包括协调轮数、子角色迭代次数、工具超时、单轮动作数量、重复工具检测和无进展终止。若编排过程超过边界，系统不会继续无限推理，而是根据当前 MASState 生成保守回答，说明已有证据、无法确认的部分和建议下一步。对于工具调用失败，服务会根据错误类型判断是否重试、换用其他工具、请求用户补充信息或降级回答。对于确认流，服务在写工具返回待确认状态后停止继续执行，并把当前工作流检查点返回给后端。

多智能体编排实现。 服务在一次任务中创建协调器、规划器、探索器、执行器和验证器等角色实例。协调器先执行意图判断，随后根据状态决定下一阶段。如果需要规划，规划器生成调查步骤和候选工具；如果需要取证，探索器调用只读工具并把事实写入观测结果；如果需要执行，执行器根据目标动作生成工具调用并进入确认流程；如果证据基本充分，验证器对结论和执行结果进行复核；最后由协调器组织面向用户的回答。角色之间不直接共享自由文本，而是通过 MASState 中的结构化字段交换计划、证据、动作和验证意见。

工具体系与平台集成实现。 当前系统维护 95 个可绑定工具，其中 69 个为只读自动工具，26 个为需要确认的写操作工具，另有 1 个内部工具用于系统流程。只读工具覆盖作业详情、事件、日志、诊断上下文、指标查询、队列状态、镜像列表、配额检查、健康概览、集群节点、存储卷、网络状态、Kubernetes 资源、Prometheus 查询、Harbor 检查、分布式训练诊断和审批历史。写操作覆盖作业重提、停止、删除、交互式环境创建、训练作业创建、镜像构建管理、镜像授权、节点隔离与恢复、Pod 删除、工作负载重启、审计项处理、批量停止、用户通知、Kubernetes 扩缩容、节点标签、节点污点和管理员命令。

工具执行采用组合执行器。对于平台已有业务能力，工具请求经由 Go 后端执行，由后端完成鉴权、数据库查询、Kubernetes 客户端调用、镜像仓库访问和审计记录。对于更接近集群观测的数据，智能体服务可以通过本地适配器访问 Kubernetes API、Prometheus、Harbor、网络诊断和 GPU 诊断能力。对于存储相关任务，工具体系通过 PVC、事件、容量概览和作业挂载关系间接反映 Ceph 等存储系统状态。对于离线评测，工具执行器可以切换为快照方式，返回预先记录的结果，从而复现实验场景。

表 4.2 列出了工具体系与平台能力的对应关系。

工具类别	代表能力	主要平台来源	作用
作业诊断工具	作业详情、事件、日志、诊断上下文	Crater 后端与 Kubernetes	判断失败原因和生命周期状态
指标与队列工具	作业指标、实时容量、队列状态、配额	Prometheus 与调度信息	分析排队、资源不足和性能异常

工具类别	代表能力	主要平台来源	作用
镜像工具	镜像列表、构建详情、授权信息	Crater 后端与 Harbor	支撑镜像选择和构建排查
存储与网络工具	PVC、容量、网络摘要、RDMA 状态	Kubernetes、存储系统、节点诊断	分析挂载失败、容量不足和分布式训练异常
管理治理工具	集群健康、节点详情、审计项、空跑检测	Crater 管理端与集群状态	支撑管理员巡检和治理
写操作工具	停止、重提、删除、节点操作、审计处理	Crater 后端与 Kubernetes	在确认后改变平台状态
评测与内部工具	运行摘要、内部报告保存	智能体服务与数据库	支撑评估、巡检和审计闭环

工具 Schema 注册以统一声明为基础。下面给出一个简化示例，展示工具声明需要包含的核心字段。正式实现中，工具声明会被转换为模型可识别的工具描述，并与执行适配器建立映射。

```
工具名: query_job_metrics
用途: 查询作业的 GPU、CPU 和内存指标
参数:
  job_name:
    类型: string
    必填: true
    含义: 作业系统名称
  metrics:
    类型: list[string]
    必填: false
    含义: 指标名称列表
  time_range:
    类型: string
    必填: false
    默认值: last_2h
返回:
  status: 执行状态
  result: 指标聚合结果
风险等级: 只读
允许角色: 探索器, 验证器, 协调器
执行适配: Prometheus 查询或后端聚合接口
```

写操作工具的 Schema 会额外声明风险摘要、确认交互和可修改参数。执行器发起写操作后，工具执行器返回待确认状态，后端生成确认记录，前端展示参数表单。用户确认后，后端根据确认结果执行真实操作，并把结果写回工具调用记录。智能体服务恢复后，不再重复发起同一写工具，而是读取确认结果并进入验证或总结阶段。

质量评估实现。 系统实现了在线反馈、会话质量评估和离线评测支撑。在线反馈用于收集用户对回答的正负反馈和维度评分；会话质量评估基于消息、工具调用和运行事件，对回答相关性、事实准确性、工具使用、风险控制 and 可用性进行分析；离线评测则使用固定场景、期望工具序列、期望确认动作和标准答案要点复现任务处理过程。评估结果写入质量评估记录，并可在管理端查看。

质量评估模块与工具轨迹密切相关。一个回答即使表面合理，也可能没有调用必要工具，或在没有确认的情况下声称完成了写操作。通过保存工具调用和角色事件，系统可以判断回答是否真正基于平台证据。对于复杂诊断任务，评估重点在根因是否命中、证据链是否完整、是否遗漏关键工具；对于受控执行任务，评估重点在确认流程是否完整、执行结果是否可追踪、最终回答是否准确表达操作状态。

图 4.4 总结了智能体服务内部结构。该图与第三章的抽象组件对应，但表达的是工程实现中的模块关系。

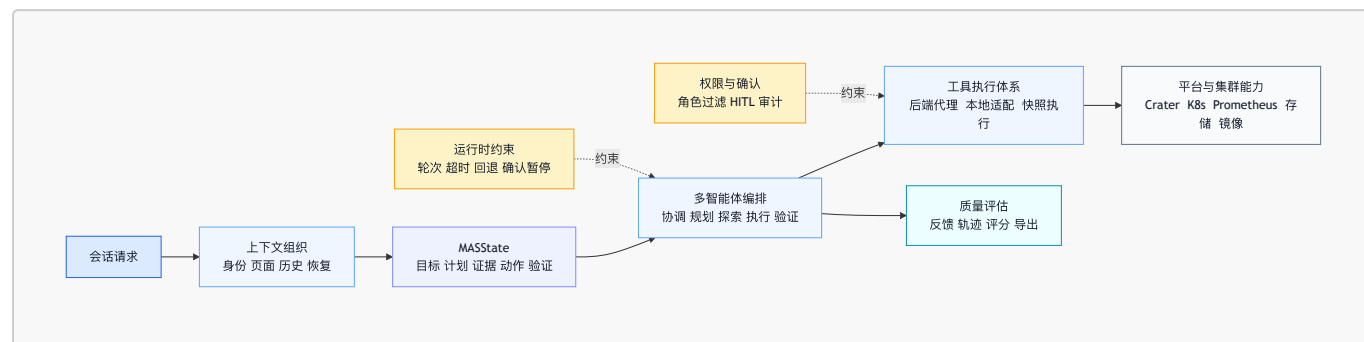


图 4.4 智能体服务内部实现结构

4.4 本章小结

本章说明了 Mops 在 Crater 智算平台中的系统实现。首先，本文介绍了 Crater 已有的平台能力，并明确 Mops 是在既有平台基础上的智能体插件式接入，而不是替代平台主体功能。其次，本文给出前端、Go 后端、Python 智能体服务、数据库和底层平台能力之间的整体架构，说明系统如何通过事件流完成对话展示，通过确认卡片处理高风险操作，通过后端代理保持权限和审计边界。再次，本文介绍了智能体服务内部的上下文组织、多智能体编排、运行时约束、工具体系、平台适配和质量评估实现，并说明工具如何与 Kubernetes API、Prometheus、存储网络、镜像仓库和 Crater 后端交互。

第四章的重点是工程落地。它对应第三章的抽象设计，将记忆机制实现为会话上下文和 MASState，将工具机制实现为统一声明、角色过滤和组合执行器，将安全机制实现为权限校验、确认恢复和审计记录，将评估机制实现为反馈、轨迹记录和质量评估。通过这种实现方式，Mops 能够在真实智算平台中处理用户帮助、状态查询、故障诊断、资源治理、受控执行、审批评估和巡检报告等任务，并为后续实验评估提供可复现的过程数据。